

Static Verification of AWS Step Functions with Sound Data-Flow Analysis and Workflow Semantics

Technical Report: Formal Development and Proofs

Anonymous Author(s)

Abstract

This technical report is the formal companion to the paper *Static Verification of AWS Step Functions with Sound Data-Flow Analysis and Workflow Semantics* [1], which presents STEP-CHECK, a static verifier for the Amazon States Language (ASL). The paper states three guarantees and gives their intuition; here we give the complete development behind them: the abstract interpretation underlying the sound data-flow provenance analysis (domain, concretization, transfer functions, fixpoint, and the termination/convergence guard), the workflow type discipline for contracts and tpestate, and the downstream-aware Saga-compensation check, with full proofs of all three theorems. Theorems 1–3 below are numbered to match Theorems 1–3 of the main paper. We also give the SC4011 algorithm in full and note the conservative basis of the concurrency and temporal analyses. Section and code references (e.g. SC1101) follow the main paper; this report is released with the artifact and is self-contained with respect to the formal development, deferring only the empirical material (the execution oracle and the evaluation) to the main paper.

Contents

1	Scope and relation to the main paper	2
2	The data-flow abstract interpretation	2
2.1	The concrete property	2
2.2	Abstract domain	2
2.3	Concretization	3
2.4	Transfer functions	3
2.5	The fixpoint	5
2.6	Reference checking	5
2.7	Nested machines	6
2.8	The dead-guard refinement (SC1110)	7
3	The workflow type discipline	7
3.1	Signatures, judgment, and validity	7
4	Downstream-aware Saga compensation	8
4.1	Model of failure and the post-commit region	8
4.2	The SC4011 algorithm	9
4.3	Reverse-order chains	10
5	Conservative analyses: concurrency and temporal (notes)	11
6	Artifact and implementation notes	11

1 Scope and relation to the main paper

The main paper keeps theorem statements and one-paragraph intuitions inline and defers their proofs and the supporting formalism here, so the body stays within venue limits. This report adds nothing to the claims; it discharges them.

We assume the ASL data plane as fixed in the paper’s background [8]. A workflow is a state machine over eight state types (`Task`, `Choice`, `Parallel`, `Map`, `Wait`, `Pass`, `Succeed`, `Fail`). A single JSON document is threaded between states and reshaped by the five-stage I/O pipeline `InputPath` $\rightarrow$ `Parameters/ItemSelector` $\rightarrow$ `ResultSelector` $\rightarrow$ `ResultPath` $\rightarrow$ `OutputPath`. A `.$` field reference reads a `JSONPath` [6] into the current document, and the runtime *fails* the execution if such a reference resolves to an absent field.

The development below is faithful to the implementation (module `passes/dataflow.rs` for Section 2, `passes/contract.rs` and `passes/typestate.rs` for Section 3, and `passes/compensation.rs` for Section 4); the execution oracle (`concrete.rs`) that empirically witnesses Theorem 1 is described in the main paper. Nested machines (`Map` iterators and `Parallel` branches) are treated in Remark 1.

2 The data-flow abstract interpretation

The provenance analysis is a forward abstract interpretation [2] over a lattice of JSON *document shapes*: it over-approximates, at each state, the set of documents with which the state may be entered, and flags a `.$` reference that is absent on every such document.

2.1 The concrete property

Fix a workflow with state set S . A concrete execution threads a JSON document; write $\mathcal{D}$ for the set of JSON values. For a state s , let $\llbracket s \rrbracket$ be the set of documents with which s is entered over all executions that reach s . A path $p = k_1.k_2.\dots.k_m$ is *present* in a document d if d is an object with key k_1 whose value has k_2 , and so on to k_m ; otherwise p is *absent*. The analysis answers a single question soundly: *is p absent in every $d \in \llbracket s \rrbracket$?* If so, a `.$` reference to p evaluated at s fails on every execution, and STEP CHECK reports SC1101.

2.2 Abstract domain

A *document shape* abstracts a set of documents by the field paths that *may* be present.

Definition 1 (Shapes). The set of shapes Σ is generated by

$$\sigma ::= \top \mid \text{Obj } r, \quad r : \text{Key} \rightarrow \Sigma \text{ a finite partial map.}$$

$\top$ is opaque (any field may be present). `Obj` r constrains an object to the keys in $\text{dom}(r)$: every top-level key that *may* occur is listed, so a key outside $\text{dom}(r)$ is present on no represented *object*. The abstraction functions send a scalar to `Obj` $\emptyset$ (no fields) and an array to $\top$ (its elements are unconstrained); this is a choice of the abstraction, not a claim that $\gamma(\text{Obj } r)$ contains only objects—indeed $\gamma(\text{Obj } r)$ admits every non-object (Definition 4), so `Obj` r is a constraint on the object case alone.

Definition 2 (Order and join). The join $\sqcup$ is

$$\top \sqcup \sigma = \sigma \sqcup \top = \top, \quad \text{Obj } r_1 \sqcup \text{Obj } r_2 = \text{Obj } r, \quad \text{dom}(r) = \text{dom}(r_1) \cup \text{dom}(r_2),$$

where $r(k) = r_1(k) \sqcup r_2(k)$ if $k \in \text{dom}(r_1) \cap \text{dom}(r_2)$, and $r(k) = r_i(k)$ if k occurs only in r_i . The induced order is $\sigma_1 \sqsubseteq \sigma_2 \iff \sigma_1 \sqcup \sigma_2 = \sigma_2$; equivalently $\sigma \sqsubseteq \top$ for all σ , and $\text{Obj } r_1 \sqsubseteq \text{Obj } r_2$ iff $\text{dom}(r_1) \subseteq \text{dom}(r_2)$ and $r_1(k) \sqsubseteq r_2(k)$ for all $k \in \text{dom}(r_1)$.

Intuitively a shape is *higher* when it admits *more* documents: $\top$ is the top, $\text{Obj } \emptyset$ the most precise object shape, and the join is the least upper bound. The join is exactly the key-union of the implementation (clone the left record, then for each key of the right record join or insert).

Definition 3 (Definite absence). A path $p = k_1 \dots k_m$ is *definitely absent* in σ if descending σ by $k_1, \dots$ reaches some $\text{Obj } r$ with $k_i \notin \text{dom}(r)$ *before* encountering $\top$. Descent stops at $\top$: a path under $\top$ is never definitely absent.

2.3 Concretization

Definition 4 (Concretization γ). $\gamma : \Sigma \rightarrow \mathcal{P}(\mathcal{D})$ is

$$\gamma(\top) = \mathcal{D}, \quad \gamma(\text{Obj } r) = \{ d \in \mathcal{D} \mid d \text{ object} \Rightarrow \forall k \in \text{keys}(d). k \in \text{dom}(r) \wedge d(k) \in \gamma(r(k)) \}.$$

A non-object d (scalar, array) has no top-level keys, so it lies in $\gamma(\text{Obj } r)$ for every r .

Lemma 1 (Absence is captured). *If p is definitely absent in σ , then p is absent in every $d \in \gamma(\sigma)$.*

Proof. Induction on $|p|$. If $p = k_1 \dots$ and $\sigma = \text{Obj } r$ with $k_1 \notin \text{dom}(r)$, then by definition of γ no $d \in \gamma(\text{Obj } r)$ has top-level key k_1 , so p is absent in d . Otherwise $k_1 \in \text{dom}(r)$ and $k_2 \dots k_m$ is definitely absent in $r(k_1)$; for any $d \in \gamma(\text{Obj } r)$, either d lacks k_1 (so p absent) or $d(k_1) \in \gamma(r(k_1))$, and by the induction hypothesis $k_2 \dots k_m$ is absent in $d(k_1)$, so p is absent in d . Definite absence cannot pass through $\top$, so the case $\sigma = \top$ never arises along the descent. $\square$

Lemma 2 (γ is monotone, $\sqcup$ is sound). $\sigma_1 \sqsubseteq \sigma_2 \Rightarrow \gamma(\sigma_1) \subseteq \gamma(\sigma_2)$, and $\gamma(\sigma_1) \cup \gamma(\sigma_2) \subseteq \gamma(\sigma_1 \sqcup \sigma_2)$.

Proof. Monotonicity is by induction on shape structure: if $\sigma_2 = \top$ it is immediate; for $\text{Obj } r_1 \sqsubseteq \text{Obj } r_2$ we have $\text{dom}(r_1) \subseteq \text{dom}(r_2)$ and $r_1(k) \sqsubseteq r_2(k)$, so any d admitted by $\text{Obj } r_1$ has $\text{keys}(d) \subseteq \text{dom}(r_1) \subseteq \text{dom}(r_2)$ and $d(k) \in \gamma(r_1(k)) \subseteq \gamma(r_2(k))$ by the induction hypothesis. Soundness of $\sqcup$ follows since $\sigma_i \sqsubseteq \sigma_1 \sqcup \sigma_2$ and γ is monotone. $\square$

2.4 Transfer functions

Each stage is a monotone function on shapes. We write $\text{narrow}(\sigma, \pi)$ for re-rooting along a path expression π (used by `InputPath/OutputPath`), $\text{ctor}(v)$ for the record built from a constructor value v (used by `Parameters/ItemSelector/ResultSelector`), and $\text{place}(\sigma, \rho, \sigma_r)$ for merging a result shape σ_r at a `ResultPath` ρ .

Re-rooting. Write ε for the empty (absent) path and $k :: \text{rest}$ for the split of a path into its first key and remainder. Then $\text{narrow}(\sigma, \varepsilon) = \sigma$ (no path) and $\text{narrow}(\sigma, \$) = \sigma$ (the bare root); $\text{narrow}(\sigma, \text{null}) = \text{Obj } \emptyset$, sound because ASL maps an `InputPath/OutputPath` of `null` to the empty object $\{\} \in \gamma(\text{Obj } \emptyset)$. For a simple path $\pi = k_1 \dots k_n$, narrow descends via nav : $\text{nav}(\top, _) = \top$ and $\text{nav}(\text{Obj } r, k :: \text{rest}) = \text{nav}(r(k), \text{rest})$ if $k \in \text{dom}(r)$, and $= \top$ otherwise (a missing intermediate key yields $\top$, over-approximating). Any path outside the simple-path fragment (wildcards, filters, functions, brackets, the `$$` context) yields $\top$; a non-string, non-`null` path value is treated as the identity ($\text{narrow}(\sigma, \cdot) = \sigma$), a case ASL deploy-time validation rejects and which is therefore unreachable in practice.

Constructors and literals. Two builders recurse over a JSON value v . The *constructor* $\text{ctor}(v)$ models a payload template (`Parameters/ItemSelector/ResultSelector`): for an object it builds $\text{Obj } r$ where a key “ $k.\$$ ” contributes $r(k) = \top$ (it *reads* an opaque value into field k), a static key k contributes $r(k) = \text{ctor}(v(k))$, a scalar leaf is $\text{Obj } \emptyset$, and an array is $\top$. The *literal* builder $\text{lit}(v)$ models verbatim data (a `Pass Result`), which ASL copies uninterpreted: it agrees

with *ctor* *except* that it does no $\cdot \$$ stripping—a key literally named “ $k.\$$ ” stays a key of that exact name. Distinguishing the two is necessary for soundness: for $\text{Result} = \{\text{“a.\$”}: v\}$ the concrete document has the top-level key “a. $\$$ ”, which is *not* in $\text{dom}(\text{ctor}) = \{a\}$, so *ctor* would under-approximate the key set.

Result and merge. The result shape σ_{res} of a state, built from its effective input eff , is: $\top$ for an opaque **Task/Map/Parallel** result; $\text{ctor}(\text{ResultSelector})$ when a **ResultSelector** pins it; and for a **Pass**, $\text{lit}(\text{Result})$ if it carries a literal **Result**, else $\text{ctor}(\text{Parameters})$ if it carries only **Parameters**, else eff itself (a bare **Pass** forwards its effective input unchanged). Merging is

$$\text{place}(\sigma, \rho, \sigma_r) = \begin{cases} \sigma_r & \rho = \text{Default or } \rho = \$ \quad (\text{result replaces the document}), \\ \sigma & \rho = \text{Discard (null)} \quad (\text{result dropped, input kept}), \\ \text{place}_{\text{in}}(\sigma, \pi, \sigma_r) & \rho = \text{Path}(\pi), \end{cases}$$

where place_{in} descends π , cloning each record and *preserving siblings*, and writes σ_r at the leaf; a $\top$ parent stays $\top$, and an unparseable π yields $\top$. (This is why a field can survive a task that never mentions it: a sub-path merge leaves the rest of the document intact.)

State transfer and error edges. For a state with entry shape σ , its output shape is

$$\text{out}(\sigma) = \begin{cases} \top & \text{JSONata mode (opaque query),} \\ \text{narrow}(\text{place}(\sigma, \rho, \sigma_{\text{res}}), \text{OutputPath}) & \text{Task/Pass/Map/Parallel,} \\ \text{narrow}(\text{eff}, \text{OutputPath}) & \text{otherwise,} \end{cases}$$

with $\text{eff} = \text{narrow}(\sigma, \text{InputPath})$ the **InputPath**-selected input from which the result shape σ_{res} is built, and ρ the state’s **ResultPath**. The result is merged into the *raw* entry shape σ , *not* eff —exactly as ASL writes **ResultPath** into the pre-**InputPath** document, so a field the **InputPath** selection dropped still survives. A **Catch** edge from a state with entry σ propagates $\text{catch}(\sigma, \rho) = \text{place}(\sigma, \rho, \top)$: the error object is opaque ($\top$), merged at the catch’s **ResultPath**. A **Retry** re-executes the same state on the same entry document, so it leaves entry shapes unchanged and the analysis ignores it.

Lemma 3 (Local soundness). *Writing a subscript c for the concrete counterpart of each stage, every transfer is sound on its own arguments:*

(re-root) $d \in \gamma(\sigma) \Rightarrow \text{narrow}_c(d, \pi) \in \gamma(\text{narrow}(\sigma, \pi));$

(build) *the concrete payload built over a document d from a template v lies in $\gamma(\text{ctor}(v))$, and from a literal v in $\gamma(\text{lit}(v))$;*

(merge) $d \in \gamma(\sigma) \wedge d_r \in \gamma(\sigma_r) \Rightarrow \text{place}_c(d, \rho, d_r) \in \gamma(\text{place}(\sigma, \rho, \sigma_r));$

(state/catch) $d \in \gamma(\sigma) \Rightarrow \text{out}_c(d) \in \gamma(\text{out}(\sigma))$ and $\text{catch}_c(d) \in \gamma(\text{catch}(\sigma, \rho)).$

Proof. By cases. *Re-rooting:* if $\sigma = \top$ or π is non-simple, $\text{narrow}(\sigma, \pi) = \top$ and $\gamma(\top) = \mathcal{D}$; for a simple path, nav keeps $d(k) \in \gamma(r(k))$ by Definition 4, and a missing intermediate key yields $\top$ (vacuously sound); $\text{narrow}(\sigma, \text{null}) = \text{Obj } \emptyset$ is sound because the selected concrete value is the empty object $\{\}$, which has no keys and hence lies in $\gamma(\text{Obj } \emptyset)$. *Constructor:* the concrete payload object has top-level keys exactly the template’s keys $\subseteq \text{dom}(\text{ctor}(v))$; a $\cdot \$$ key binds a runtime-read value in $\gamma(\top)$, and a static key’s value lies in $\gamma(\text{ctor}(v(k)))$ by induction. *Literal:* the concrete value is v copied verbatim, whose keys are exactly $\text{dom}(\text{lit}(v))$ —including any literal “ $\cdot \$$ ” key—and each child lies in the corresponding γ by induction. *Result:* an opaque result lies in $\gamma(\top)$; a pinned **ResultSelector** reduces to the constructor case; a bare **Pass** forwards eff ,

and $d \in \gamma(\text{eff})$ by hypothesis. *Merge*: $\rho \in \{\text{Default}, \$\}$ replaces, and the concrete output is the result, which lies in $\gamma(\sigma_r)$; $\rho = \text{Discard}$ keeps the input; for $\rho = \text{Path}(\pi)$, place_{in} writes σ_r at π and preserves siblings, matching the concrete merge, and a $\top$ parent (or unparseable π) stays $\top$ (sound). *State transfer* composes the above; *JSONata mode* yields $\top$. *Catch* places $\top$ (the error object) at ρ , sound since $\gamma(\top) = \mathcal{D}$. $\square$

2.5 The fixpoint

The analysis maintains a *partial* map $\Phi : S \rightarrow \Sigma$, seeding only the start state with σ_0 ; a state acquires a shape the first time an edge reaches it, and a state never reached stays outside $\text{dom}(\Phi)$ and is read as $\top$ at check time (an unreached state constrains nothing, and reporting against $\top$ never fires). One *round* updates, for every $s \in \text{dom}(\Phi)$ with current entry $\Phi(s)$: for each normal successor t , $\Phi(t) \leftarrow \Phi(t) \sqcup \text{out}(\Phi(s))$ (inserting $\text{out}(\Phi(s))$ when $t \notin \text{dom}(\Phi)$); for each **Catch** target u with result path ρ , $\Phi(u) \leftarrow \Phi(u) \sqcup \text{catch}(\Phi(s), \rho)$. Updates are in place (Gauss–Seidel) in an unspecified state order; the least fixpoint, once reached, is independent of that order. Rounds repeat until a round changes nothing.

Lemma 4 (Convergence and the soundness guard). *The forward iteration is monotone: each $\Phi(s)$ only ascends under $\sqsubseteq$. On the bounded-depth fragment—workflows in which every result shape σ_{res} is static (built by *ctor/lit* from constructor text, or opaque $\top$), so that no state both selects a sub-document and re-embeds it under a **ResultPath**—the iteration reaches the least fixpoint. In general the domain admits infinite ascending chains, so convergence is not guaranteed; the implementation therefore caps the loop at $N = |S| \cdot (\kappa_0 + 2) + 2$ rounds (with κ_0 the implementation’s key count and $|S|$ the state count) and, on exceeding the cap, sets a non-converged flag that suppresses all SC1101 and SC1110 reporting for that (sub-)machine. A finding is thus emitted only at a genuine fixpoint: the cap can cost precision but never soundness.*

Proof. Every update is a join, so each $\Phi(s)$ is non-decreasing (Lemma 2). Let κ be the size of the workflow’s *key universe*: the finite set of keys occurring in any constructor (**Parameters**, **Result**, **ResultSelector**, **ItemSelector**), in the declared input schema, or as a segment of a **ResultPath** or catch **ResultPath**. On the bounded-depth fragment there is a bound D on the depth of every reachable shape: *ctor/lit* build records whose nesting is bounded by the static constructor text; place_{in} writes a static σ_{res} at the fixed depth $|\pi|$ of its **ResultPath** and *overwrites* rather than stacks; and *narrow*, *nav*, and $\sqcup$ never increase depth. Every reachable shape then lies in the (finite-height) lattice of records of depth at most D over the κ -key universe; a non-decreasing chain in a finite-height lattice stabilises, so the iteration reaches the least fixpoint. Outside the fragment a bare **Pass** can forward its narrowed input as σ_{res} while a **ResultPath** re-embeds that sub-document one level deeper; a cycle through such a state deepens shapes without bound, so no finite-height bound exists and the iteration need not converge. The implementation’s cap N uses a key count κ_0 that omits **ResultPath** segments, making N a heuristic guard rather than a tight bound (in debug builds an assertion additionally flags the event). Because reporting is gated on the converged flag, exceeding N merely suppresses findings; soundness (Theorem 1) is therefore unconditional, whereas convergence—and hence the possibility of *any* finding—is guaranteed only on the bounded-depth fragment. $\square$

2.6 Reference checking

At the fixpoint—and only if the machine converged (Lemma 4)—for each state s with $\text{eff} = \text{narrow}(\Phi(s), \text{InputPath})$ the analysis collects the set $\text{refs}(s)$ of paths the runtime evaluates and aborts on when absent: the $.\$$ keys of **Parameters** and **ItemSelector** (at any depth), and a **Map**’s **ItemsPath** string. It excludes the references it does not interpret—any string not beginning with $\$$, the context object $$$$, **States.*** intrinsics, the bare root $\$$, and any path outside the simple-path fragment—and skips *all* references of a JSONata state (whose $\{\% \%$ reads are not

JSONPath). For a surviving p it reports SC1101 iff p is definitely absent in eff (the lookup returns **Missing**; a path into a $\top$ or otherwise unconstrained region returns **Maybe** and is never flagged). It *does not* inspect **Choice** operands; see Section 2.8.

Assumption 1 (Runtime resolution on non-objects). The ASL JSONPath engine resolves a dotted reference $k_1 \dots k_m$ by successive object-key lookup: applying a key to a non-object (scalar or array) fails with a path-not-found error, and a numeric segment denotes a property name rather than an array index [8, 6]. Under this assumption the concrete *present* relation of Section 2 (object-key descent) coincides with runtime resolution, and γ 's admission of arbitrary non-objects under **Obj** r is sound: a reference descending into a non-object simply fails, exactly as “absent” predicts.

Theorem 1 (No false positives). *If STEP CHECK reports SC1101 for a path p at a state s , then in every concrete execution that reaches s , p is absent from the state document, so the ASL runtime fails the reference. Equivalently, the analysis never flags a reference some execution could satisfy.*

Proof. First, the fixpoint over-approximates the reachable documents: for every s and every $d \in \llbracket s \rrbracket$, $d \in \gamma(\Phi(s))$. We argue by induction on the length of the execution prefix reaching s . *Base:* the start seed σ_0 satisfies $\llbracket s_0 \rrbracket \subseteq \gamma(\sigma_0)$ —in the native tier $\sigma_0 = \top$ and $\gamma(\top) = \mathcal{D}$; in the typed tier $\sigma_0 = \text{Obj } r_0$ with $\text{dom}(r_0)$ the declared input fields, and the claim holds under the closed-world reading of the declared contract (the input contains no field outside the schema). *Step:* suppose s is reached by a transition from s' . If it is a normal transition, the concrete document is $\text{out}_{\text{concrete}}(d')$ for some $d' \in \llbracket s' \rrbracket$; by the induction hypothesis $d' \in \gamma(\Phi(s'))$, by Lemma 3 $\text{out}_{\text{concrete}}(d') \in \gamma(\text{out}(\Phi(s')))$, and since the round joined $\text{out}(\Phi(s'))$ into $\Phi(s)$, by Lemma 2 $\text{out}_{\text{concrete}}(d') \in \gamma(\Phi(s))$. If it is a **Catch** transition, the concrete document is the input merged with an (arbitrary) error object; *catch* places $\top$ at the catch path, which admits every such document, and the round joined it into $\Phi(s)$, so again the document lies in $\gamma(\Phi(s))$. The **InputPath** narrowing applied at s is itself sound (Lemma 3), so the *effective* entry documents lie in $\gamma(eff)$.

Now suppose SC1101 fires for p at s . Then p is definitely absent in eff , and by Lemma 1 no $d \in \gamma(eff)$ contains p . Since every effective entry document at s lies in $\gamma(eff)$, p is absent on all of them, so by Assumption 1 the $.\$$ reference to p fails on every execution reaching s . A finding is emitted only when the machine converged (Lemma 4), i.e. at a genuine fixpoint, so the conclusion holds whenever SC1101 is reported. $\square$

Corollary 1 (Native unconditionality). *On the native tier ($\sigma_0 = \top$), Theorem 1 holds for an arbitrary execution input. On the typed tier it holds relative to the closed-world reading of the declared input schema; where inputs are open, the native tier is the sound fallback. (The typed seed also admits a second closed-world source: when a workflow declares no input schema but its start task declares an input contract, the start seed is that contract's field record.)*

2.7 Nested machines

A **Map**'s iterator and a **Parallel**'s branches are themselves state machines, each analysed by the same procedure under a seed derived from the enclosing state's entry.

Remark 1 (Nested-machine seeds). The nested analysis runs—crucially for soundness—only when the enclosing machine has converged (Lemma 4); otherwise its entry shapes are under-approximate and the recursion is suppressed together with the enclosing machine's own reporting. Each **Map** iteration is seeded with $\text{ctor}(\text{ItemSelector})$ when an **ItemSelector** is declared and $\top$ otherwise; each **Parallel** branch is seeded with $\text{ctor}(\text{Parameters})$ when the state declares **Parameters** and with the effective input $eff = \text{narrows}(\Phi(s), \text{InputPath})$ otherwise—exactly

the document ASL hands each branch. Each seed over-approximates the corresponding per-iteration/per-branch document (Lemma 3), so Theorem 1 extends unconditionally to findings emitted inside a nested machine.

2.8 The dead-guard refinement (SC1110)

The same shapes support a path-sensitive *warning*. Unlike a `.$` payload reference, a **Choice** comparator is a *guard* evaluated in rule order: the runtime tries each rule until one matches and follows its branch. A value comparator on an unresolvable operand does not silently return false—ASL raises a **States.Runtime** error [8], which is precisely why **IsPresent** exists. We nonetheless report only a *warning*, not an SC1101 error, for two reasons: the rule may never be reached (an earlier rule can match first and divert control), so we cannot promise a runtime abort; and the finding’s real content is that a branch whose guard *requires* a definitely absent field can never be taken—dead logic. SC1110 fires when a **Choice** rule’s condition requires presence of a field definitely absent in the state’s effective (post-**InputPath**) shape. Presence is required by value comparators, **IsNull**, the type predicates, and **IsPresent:true**; it is *not* required by **IsPresent:false**. The check is conservative on Boolean structure: a conjunction is dead if any conjunct is dead, a disjunction only if *all* disjuncts are dead, and a negation is never flagged. Soundness of the underlying “definitely absent” judgment is exactly Lemma 1.

3 The workflow type discipline

3.1 Signatures, judgment, and validity

Let Γ collect the declared task signatures. Each task T has a *contract* $A \rightarrow B$ (it consumes the field-record A and produces B) and a *typestate* [4, 5] transition $s_{\text{in}}^T \rightarrow s_{\text{out}}^T$ over business states S_B ; we bundle the two into one signature $T : (A, s_{\text{in}}^T) \rightarrow (B, s_{\text{out}}^T)$. The well-typedness judgment $\Gamma \vdash W : (A, s) \Rightarrow (B, s')$ carries a *pair* type—a field-record and a business state—and is generated by:

$$\begin{array}{c} \frac{(T : (A, s_{\text{in}}) \rightarrow (B, s_{\text{out}})) \in \Gamma}{\Gamma \vdash T : (A, s_{\text{in}}) \Rightarrow (B, s_{\text{out}})} \text{ (TASK)} \quad \frac{\Gamma \vdash T : (A, s) \Rightarrow (B, s') \quad A \subseteq A'}{\Gamma \vdash T : (A', s) \Rightarrow (B, s')} \text{ (SUB)} \\[10pt] \frac{\Gamma \vdash T_1 : (A, s) \Rightarrow (B, s') \quad \Gamma \vdash T_2 : (B, s') \Rightarrow (C, s'')}{\Gamma \vdash T_1; T_2 : (A, s) \Rightarrow (C, s'')} \text{ (SEQ)} \\[10pt] \frac{\Gamma \vdash T_1 : (A, s) \Rightarrow (B, s') \quad \Gamma \vdash T_2 : (A, s) \Rightarrow (B, s')}{\Gamma \vdash \text{Choice}(T_1, T_2) : (A, s) \Rightarrow (B, s')} \text{ (CHOICE)} \end{array}$$

The width-subsumption rule (SUB) widens the *available* input fields—a task needing A still runs when more fields $A' \supseteq A$ are present—and leaves the business state untouched. It is what lets the field component be matched by *containment* while the typestate component is matched by *equality*, mirroring the two checks below; without it the SEQ rule would force the middle field-record to match exactly, which the tool does not require.

The judgment is read over the workflow’s *task-adjacency* structure: STEP CHECK projects the control-flow graph onto its **Task** states, following *normal* edges through the *transparent* states (**Choice**, **Pass**, **Wait**) that carry no contract, so $T_1; T_2$ denotes a task immediately followed—through any transparent states—by another and **Choice**(T_1, T_2) a branch whose arms rejoin. **Catch** (error) edges are *not* projected, and **Map/Parallel** states are opaque adjacency endpoints whose sub-machines are checked separately, so the discipline governs the normal-flow task graph only. A declared *protocol* $\delta \subseteq S_B \times S_B$ lists the legal business-state transitions, and $\text{Valid}(W) \iff \forall T \in W. (s_{\text{in}}^T, s_{\text{out}}^T) \in \delta$.

The checks are edge-local. For each task-adjacency $T_1 \rightarrow T_2$ whose endpoints both carry the relevant declarations: the typed-contract check SC1010 fires iff some required field of T_2 is missing from T_1 's produced fields ($in(T_2) \not\subseteq out(T_1)$); the typestate check SC2001 fires iff $s_{out}^{T_1} \neq s_{in}^{T_2}$; and SC2002 fires iff $(s_{in}^{T_2}, s_{out}^{T_2}) \notin \delta$ when a non-empty protocol is declared. A **Choice** imposes the predecessor's premise on each branch head. An edge missing an endpoint declaration is passed over—the checks are vacuous there—so the guarantee below is stated for *fully declared* workflows.

Theorem 2 (Declared-tier consistency). *Let W be fully declared: every **Task** carries a contract and both typestate labels, and a protocol δ is declared. Then SC1010, SC2001, and SC2002 emit no finding iff*

$$\begin{aligned} & in(T_2) \subseteq out(T_1) \text{ and } s_{out}^{T_1} = s_{in}^{T_2} \text{ for every adjacency } T_1 \rightarrow T_2, \\ & \text{and } (s_{in}^T, s_{out}^T) \in \delta \text{ for every task } T. \end{aligned} \quad (*)$$

The three checks are therefore a sound and complete decision procedure for $(*)$: every W satisfying $(*)$ is accepted, and every violation is reported by a finding naming the offending edge (SC1010/SC2001) or task (SC2002). On the series-parallel fragment—acyclic task graphs built from sequencing and fully rejoining binary **Choice** whose arms share a common branch-point and rejoin pair type—condition $(*)$ is moreover equivalent to derivability $\Gamma \vdash W : (A, s) \Rightarrow (B, s')$ together with $\text{Valid}(W)$, so on that fragment the checks decide well-typedness in the calculus above.

Proof. The equivalence “no finding $\iff (*)$ ” is immediate: each check is exactly the negation of one clause of $(*)$, evaluated independently per edge (per task for SC2002), and full declaration leaves no edge vacuous. This holds for an arbitrary normal-flow task graph, cyclic ones included—a back-edge is simply another adjacency subject to $(*)$.

For the series-parallel fragment we show $(*) \iff \Gamma \vdash W : (A, s) \Rightarrow (B, s') \wedge \text{Valid}(W)$. ($\Leftarrow$) A derivation types each adjacency by SEQ, whose shared middle type (B, s') forces $in(T_2) \subseteq out(T_1)$ on the field component (a preceding SUB step only enlarges the available fields) and $s_{out}^{T_1} = s_{in}^{T_2}$ on the typestate component; CHOICE imposes the same shared type on both arms, extending the equalities across a branch and its rejoin. $\text{Valid}(W)$ supplies the protocol clause, giving $(*)$. ($\Rightarrow$) Assume $(*)$. Build the derivation bottom-up: TASK gives each task $T : (in(T), s_{in}^T) \Rightarrow (out(T), s_{out}^T)$. For a sequential adjacency $T_1; T_2$ take the middle type $(B, s') = (out(T_1), s_{out}^{T_1})$: by $(*)$, $in(T_2) \subseteq out(T_1) = B$ and $s_{in}^{T_2} = s_{out}^{T_1} = s'$, so SUB lifts the TASK axiom of T_2 to $(B, s') \Rightarrow (out(T_2), s_{out}^{T_2})$ and SEQ applies. A fragment **Choice** types both arms at the common branch-point type (A, s) and common rejoin type (B, s') (each arm as above), so CHOICE applies. The result derives $\Gamma \vdash W$, and the per-task protocol membership gives $\text{Valid}(W)$. Outside the fragment—non-rejoining or multiway branches, type-misaligned rejoins, or cycles—no finite derivation in this three-constructor calculus need exist even when $(*)$ holds; there the edge-condition statement is the operative one, and it is exactly what the tool decides.

Both readings are *relative to the declared annotations*, and here that scoping is exact: contracts and typestate labels are never inferred—only the idempotence/persistence/effect annotations of Section 4 are—so on this tier Γ is the declared data and the three checks are unconditionally error-level. A workflow lacking a declaration is outside the theorem's scope (the corresponding check is vacuous on that edge), not silently downgraded to a warning. $\square$

4 Downstream-aware Saga compensation

4.1 Model of failure and the post-commit region

Let W declare which tasks are *persistent* and, for each persistent P , a declared *compensator* C (the Saga pattern [3]). Every *fallible* state—a **Task**, **Map**, or **Parallel**—may raise the generic error `States.TaskFailed`; an error caught by a *broad Catch* (one whose `ErrorEquals` contains

`States.ALL` or `States.TaskFailed`) is routed to the catch target, otherwise it propagates to workflow failure. `Parallel` and `Map` are fallible black boxes; a nested persistent task is analysed within its own sub-machine by the same procedure.

The completeness half of the guarantee below is stated relative to an explicit model of execution, made precise here so the theorem is neither over- nor under-claimed.

Assumption 2 (Failure model). (A1) *Single post-commit fault*: on each execution we reason about the *first* failure after P commits, and the compensation flow it triggers is itself fault-free—a standard single-fault Saga assumption. (A2) *Path feasibility*: `Choice` guards are abstracted, so every normal edge is feasible (the analysis is path-insensitive). (A3) *Unique generic catcher*: no fallible state has two `Catch` clauses both matching `States.TaskFailed`, so the existential test over catchers coincides with ASL’s first-match resolution for the modeled error. (A4) *Linear handlers*: from a broad `Catch` target, control follows a unique normal successor until it reaches C (as the typed DSL builds compensation chains), so “reaches C ” means “executes C ”.

Definition 5 (Post-commit region). $R(P)$ is the set of states reachable from P ’s normal successors by following *normal* (non-error) control-flow edges *without passing through C* . These are exactly the states that execute after P has committed its durable effect and before any compensation.

Definition 6 (Saga-completeness). A persistent task P with compensator C is *Saga-complete* when every execution in which P commits its effect and a subsequent *post-commit failure* occurs—a fallible state in $R(P)$ raising the generic error, or a `Fail` terminal reached on a normal path in $R(P)$ —also executes C .

4.2 The SC4011 algorithm

Listing 1: Downstream-aware Saga check (per persistent task).

```
SC4011(W, P, C):
  region = post_commit_region(W, P, C)          # the region R(P)
  foreach Q in region:
    if is_fallible(Q) and not failure_reaches(Q, C):
      fire SC4011 at Q                          # a post-commit failure at Q
    escapes C
    if is_fail_terminal(Q):
      fire SC4011 at Q                          # a normal path commits P then ends
    in failure

post_commit_region(W, P, C):
  successors,
  return forward_closure(normal_successors(P),  # not passing through C
                        edges = normal, without = {C})

failure_reaches(Q, C):
  # Q's own generic failure is caught broadly and the handler reaches C by normal
  flow
  return exists c in catches(Q):
    broad(c.error_equals) and reaches(c.target, C, edges = normal)

is_fallible(Q):      return Q.kind in {Task, Map, Parallel}
is_fail_terminal(Q): return Q.kind == Fail
broad(errs):         return "States.ALL" in errs or "States.TaskFailed" in errs
reaches(s, C, e):    return C in forward_closure({s}, edges = e)
```

Theorem 3 (Compensation completeness under the declared model). *Fix a workflow W with declared persistence and compensators, and adopt Assumption 2. For a persistent task P with compensator C , SC4011 (Listing 1) emits no finding iff P is Saga-complete—iff every fallible $Q \in R(P)$ has a broad **Catch** whose target reaches C , and no normal path from P reaches a **Fail** terminal in $R(P)$. Under the model, SC4011 is a sound and complete decision procedure for Saga-completeness.*

Proof. Soundness (every finding is real, relative to A2). Suppose SC4011 fires. In the first case it fires at a fallible $Q \in R(P)$ with $\neg \text{failure_reaches}(Q, C)$. By A2 the normal path $P \rightarrow \dots \rightarrow Q$ is feasible, so some execution runs P to commitment and reaches Q ; being fallible, Q raises **States.TaskFailed** (A1: the single post-commit fault). By A3 the catcher the test inspects is the one ASL would match, and it is absent, non-broad, or routes to a handler not reaching C ; by A4 “reaches C ” is exact, so C does not execute. This is a concrete post-commit failure that leaks P ’s effect. In the second case SC4011 fires at a **Fail** terminal $Q \in R(P)$: the feasible normal path $P \rightarrow \dots \rightarrow Q$ commits P and then fails with no compensation—a more direct witness.

Completeness (every defect is found). Suppose P is *not* Saga-complete: some execution commits P and then a post-commit failure occurs without C . By A1 consider that first failure. If it is a fallible state Q , then $Q \in R(P)$; by A3 its generic error resolves to the single catcher the test inspects, and since C did not run, that catcher does not route to C (A4 makes “reaches” exact), so $\neg \text{failure_reaches}(Q, C)$ and SC4011 fires at Q . If instead the execution reaches a **Fail** terminal on a normal path without C , that terminal lies in $R(P)$ and SC4011 fires there. Conversely, if every fallible $Q \in R(P)$ has its (unique, by A3) broad catcher reaching C and no normal path bypasses C to a **Fail**, then by A4 every post-commit fault routes into a chain that runs C , so P is Saga-complete. Hence SC4011 fires exactly on the non-Saga-complete workflows.

The guarantee is *relative to the declared facts*: a compensator is never *inferred*, so under pure inference the premise is absent and SC4011 stays silent (in contrast to the name-based, warning-level SC4010). $\square$

Remark 2 (Where the assumptions bite). Relaxing Assumption 2 costs completeness, not soundness. Without A1, a fault *inside* the compensation chain (e.g. a fallible compensator) can leak an earlier effect, which SC4011 does not model. Without A3, a fallible state whose first catcher matches **States.TaskFailed** but routes away from C , shadowing a later broad catcher, is missed. Without A4, a broad catcher whose handler *may* reach C on one branch and a **Fail** on another passes the existential test yet can skip C ; deciding this exactly needs C to *postdominate* the handler, which the check approximates by reachability. Non-generic errors (**States.Timeout**, service faults) caught by narrow catchers, and cross-branch aborts of a **Parallel/Map** that strand a sibling’s committed effect, are likewise outside the model. Closing these is future work; the corpus in the main paper satisfies A1–A4.

4.3 Reverse-order chains

Saga-completeness of a single (P, C) pair is local, but a multi-step Saga must unwind *all* prior effects on a late failure. Take persistent tasks $P_1, \dots, P_n$ committed in order with compensators $C_1, \dots, C_n$ wired as a *reverse-order chain*: each C_i ’s *unique* normal successor is C_{i-1} , and C_1 ’s is the failure terminal. Under this linear-chain hypothesis—exactly A4 of Assumption 2 applied to the chain—running C_i runs the whole prefix $C_i, C_{i-1}, \dots, C_1$ in order, so applying the single-pair check of Theorem 3 to each (P_i, C_i) certifies that any post-commit failure unwinds the full committed prefix. The hypothesis is essential: without the unique-successor wiring, per-pair reachability certifies neither the *order* nor the *completeness* of the unwind—a branch in the chain could skip some C_j while still reaching a later C_k . This is how the typed DSL builds a Saga by construction, which is why a correctly built DSL Saga raises no SC4011 and a one-edge break (redirecting one compensator to the failure terminal) makes SC4011 fire on precisely the

upstream steps whose effects would then leak (the chain-break experiment in the main paper’s evaluation).

5 Conservative analyses: concurrency and temporal (notes)

The concurrency and temporal analyses are native but deliberately *conservative*: they under-approximate to avoid false alarms and make no soundness claim. All but one of their findings are warnings; the sole exception is the definite mis-configuration SC6003 (below), reported as an error.

Concurrency (SC5001/SC5002). Parallel branches and concurrent Map iterations each receive their own copy of the document, so the document cannot race; only *external* resources can. The check mines each task’s resource key (service plus the named target: `TableName`, `QueueUrl`, `TopicArn`, `Bucket`) and considers only *overwriting* writes—`PutItem`/`PutObject`, which are last-writer-wins. A task counts as a write only when it is annotated non-idempotent (or persistent): an unannotated `PutItem` is not flagged, a deliberate precision trade that keeps the analysis silent absent an effect declaration and contributes to its zero-false-positive record. The check also excludes `UpdateItem` (an atomic merge that commutes for disjoint attributes) and fire-and-forget `Publish`/`SendMessage` (independent messages). SC5001 flags two Parallel branches that overwrite the same resource; SC5002 a concurrent Map (`MaxConcurrency` $\neq 1$) whose iterations overwrite a shared resource while threading *no* per-item data (no `.$` reference in the write, so every iteration clobbers the same item). Restricting to overwriting writes with a provably shared key is what makes the analysis report a conflict only when two branches *provably* touch the same key—hence zero in-the-wild false positives, at the cost of not claiming completeness.

Temporal (SC6001–SC6003). SC6001 warns when a callback (`waitForTaskToken`) or Activity task has neither `TimeoutSeconds` nor `HeartbeatSeconds`, so it can wait up to the platform’s one-year maximum. SC6002 warns when a task’s worst-case retry budget—the geometric back-off sum $\sum_{k=0}^{a-1} interval \cdot backoff^k$ over a attempts (defaults `interval` = 1s, `backoff` = 2, $a = 3$)—exceeds a declared machine `TimeoutSeconds`. SC6003 is a definite mis-configuration: `HeartbeatSeconds` $\geq$ `TimeoutSeconds`, which the platform itself rejects only at deploy time. SC6001 is a warning because a missing timeout is occasionally acceptable for a short-lived task; SC6003 is the one error-level temporal finding, as it is unambiguous.

6 Artifact and implementation notes

This section records the implementation detail deferred from the main paper’s implementation section. Excluding tests, STEP CHECK is 4,552 lines of Rust, divided as follows: the workflow IR, 323 lines; the three frontends (ASL parse and emit, the typed DSL, and CNCF [7]), 784; annotation resolution and inference, 275; the eight analysis passes and their manager, 1,730; diagnostics, 115; the execution oracle, 269; the mutation engine, 316; and the command-line interface together with the statistics, evaluation, and oracle harnesses, 740; a further 383 lines of in-tree tests are not counted above. The data-flow pass is by far the largest analysis at 584 lines—the shape lattice of Section 2, its transfer functions, and the control-flow fixpoint—while the other seven passes range from roughly 85 to 240 lines each. (These counts are for the artifact revision and drift with continued development.)

The command-line tool exposes `check`, `infer`, `scan`, `stats`, `eval`, `oracle`, `eval-pairs`, `mutate`, `emit`, and `demo`. `check` runs the eight passes; `emit` round-trips a workflow through the ASL emitter unchanged, the control form against which the multi-validator baseline of the main

paper scores each injected defect; `oracle` runs the data-flow soundness cross-check; `eval-pairs` drives the real-bug benchmark by replaying each pre-fix and post-fix workflow; `mutate` injects one defect of a chosen class; and `demo` emits a built-in typed-DSL example to ASL.

The ASL emitter is currently lossy for a workflow’s pure I/O-processing fields: among others it does not re-emit `InputPath`, `OutputPath`, or `ResultSelector`, nor the machine-level `TimeoutSeconds`; it also drops the `QueryLanguage` directive (a JSONata machine re-emits without it) and canonicalises a `Map`’s `ItemProcessor` to the older `Iterator` form. The AWS round-trip is therefore exercised on DSL-authored workflows that use none of these fields, where the deployed state machine is behaviourally equivalent to the verified IR.

References

- [1] Anonymous Author(s). Static Verification of AWS Step Functions with Sound Data-Flow Analysis and Workflow Semantics. Under submission to ICSOC, 2026. (The main paper this report accompanies.)
- [2] P. Cousot and R. Cousot. Abstract interpretation: a unified lattice model for static analysis of programs by construction or approximation of fixpoints. In *POPL*, 1977.
- [3] H. Garcia-Molina and K. Salem. Sagas. In *ACM SIGMOD*, 1987.
- [4] R. E. Strom and S. Yemini. Typestate: a programming language concept for enhancing software reliability. *IEEE Transactions on Software Engineering*, 12(1), 1986.
- [5] R. DeLine and M. Fähndrich. Typestates for objects. In *ECOOP*, 2004.
- [6] S. Gössner, G. Normington, and C. Bormann (eds.). *RFC 9535: JSONPath — Query Expressions for JSON*. IETF, 2024.
- [7] Cloud Native Computing Foundation. *Serverless Workflow Specification*. <https://serverlessworkflow.io/>.
- [8] Amazon Web Services. *AWS Step Functions Developer Guide / Amazon States Language specification*. <https://states-language.net/spec.html>.